# Contents


**1 杂项** **1**

1.1 线下 vscode 配置 . . . 1
1.2 一些思路 . . . 1
1.3 大质数 及取模（常数小） . . . 2
1.4 内存估算 . . . 2
1.5 正负数除法上下取整 . . . 2
1.6 对拍 . . . 2
1.7 int128 输入输出流 . . . 4
1.8 随机数 rng () . . . 4
1.9 int128 输入输出 . . . 4
1.10 分数四则运算 . . . 4
1.11 基姆拉尔森公式 . . . 6
1.12 德州扑克 . . . 7


# 1 杂项

## 1.1 线下 vscode 配置

ctrl+ , 打开设置

搜 mouse 加上鼠标改编辑器和终端字体大小，

搜 smooth 把所有相关选项都打开 (美观)

自动编译运行 task, 按 ctrl+shift+B 找到 json 文件修改

```
{
    "label": "C++",
    "type": "shell",
    "command": "g++ -std=c++23 -O2 -pipe -static -s \"${file}\" -o exe && ./exe",
    "options": {
        "cwd": "${fileDirname}"
    },
    "presentation": {
        "reveal": "always",
        "focus": true,
        "panel": "shared"
    },
    "problemMatcher": "$gcc"
}
```

(改 ctrl+shift+n 的快捷键)

ctrl + shift+p 搜 open keyboard shortcuts(JSON)

```
[
    {
        "key": "Ctrl+Alt+n",
        "command": "workbench.action.tasks.runTask",
        "args": "C++"
    }
]
```

## 1.2 一些思路

- 贡献法：考虑哪些元素会被计入答案
- 遇到 $a \ mod \ b$ 可以试着转化为 $a - b \times \lfloor \frac{a}{b} \rfloor$

- 值域比较小且数组中元素各不同可以考虑调和级数 nlogn 复杂度做法

### 1.3 大质数 及取模（常数小）

```
#define int long long
int P = (1LL<<61)-1;
int mul(int a,int b){
    i128 x = (i128)a*b;
    u64 res = (u64)(x&P)+(u64)(x>>61);
    res = (res&P)+(res>>61);
    if(res>=P) res -= P;
    return (int)res;
}
```

### 1.4 内存估算

| 类型 | 字节数 | 256 MB 大约能开 |
|---|---|---|
| bool | 1 B | $2.68 * 10^8$ |
| char | 1 B | $2.68 * 10^8$ |
| short | 2 B | $1.34 * 10^8$ |
| int | 4 B | $6.71 * 10^7$ |
| long long | 8 B | $3.36 * 10^7$ |
| float | 4 B | $6.71 * 10^7$ |
| double | 8 B | $3.36 * 10^7$ |
| long double | 通常 16 B | $1.68 * 10^7$ |
| 指针 | 8 B (64 位) | $3.36 * 10^7$ |

### 1.5 正负数除法上下取整

```
int Ceil(int fz,int fm){ // 2.5 -> 3, -2.1 -> -2
    if(fm<0) fm *= -1,fz *= -1;
    return fz>=0?(fz+fm-1)/fm:fz/fm;
}
int Floor(int fz,int fm){ // 2.5 -> 2 , -2.5 -> -3
    if(fm<0) fm *= -1,fz *= -1;
    return fz>=0?fz/fm:(fz-fm+1)/fm;
}
```

### 1.6 对拍

```
#!/bin/bash

g++ -std=c++2a -O2 test.cpp -o test
# g++ -std=c++2a -O2 check.cpp -o check
g++ -std=c++2a -O2 std.cpp -o std
g++ -std=c++2a -O2 data.cpp -o data

// 普通对拍
for((i=1;;i++))
do
    ./data > in.txt
    ./test < in.txt > out_user.txt
    ./std  < in.txt > out_brute.txt
    if ! diff out_user.txt out_brute.txt > /dev/null
    then
```

```
        echo " 发现不一致! 输入如下: "
        cat in.txt
        echo "test 输出: "
        cat out_user.txt
        echo "std 输出: "
        cat out_brute.txt
        exit
    fi
    echo " 通过测试 $i"
done
// 带 check 的
for((i=1;;i++))
do
    ./data > in.txt
    cat in.txt > temp.txt
    ./test < in.txt >> temp.txt
    ./check < temp.txt > out_user.txt
    ./std  < in.txt > out_brute.txt
    if ! diff out_user.txt out_brute.txt > /dev/null
    then
        echo " 发现不一致! 输入如下: "
        cat in.txt
        echo "test 输出: "
        cat out_user.txt
        echo "std 输出: "
        cat out_brute.txt
        exit
    fi
    echo " 通过测试 $i"
done
```

### 1.7 int128 输入输出流

```
using i128 = __int128;

// 重载输入运算符以支持 __int128 类型
std::istream &operator>>(std::istream &is, __int128 &val) {
    std::string s;
    bool flag = true;
    is >> s, val = 0;
    if (s[0] == '-') flag = false, val = std::stoi(s.substr(0, 2)), s = s.substr(2);
    for (char &c: s) val = val * 10 + (c - '0') * (!flag ? -1 : 1);
    return is;
}

//重载输出运算符以支持 __int128 类型
std::ostream &operator<<(std::ostream &os, __int128 val) {
    if (val < 0) os << "-", val = -val;
    if (val > 9) os << val / 10;
    os << static_cast<char>(val % 10 + '0');
    return os;
}
```

### 1.8 随机数 rng ()

```
std::mt19937_64 rng(std::chrono::system_clock::now().time_since_epoch().count());
```

### 1.9 int128 输入输出

赛时打嫌浪费时间可以 #define int ___int128, 然后每次输入 (long long)t 然后赋值给指定的变量，最后输出结果的时候在 (long long)ans (((

```
using i128 = __int128;

// 重载输入运算符以支持 __int128 类型
std::istream &operator>>(std::istream &is, __int128 &val) {
    std::string s;
    bool flag = true;
    is >> s, val = 0;
    if (s[0] == '-') flag = false, val = std::stoi(s.substr(0, 2)), s = s.substr(2);
    for (char &c: s) val = val * 10 + (c - '0') * (!flag ? -1 : 1);
    return is;
}

//重载输出运算符以支持 __int128 类型
std::ostream &operator<<(std::ostream &os, __int128 val) {
    if (val < 0) os << "-", val = -val;
    if (val > 9) os << val / 10;
    os << static_cast<char>(val % 10 + '0');
    return os;
}
```

### 1.10 分数四则运算

```
template<class T>
struct Frac {
    T num;  //分子
    T den;  // 分母
```

```
Frac(T num_, T den_) : num(num_), den(den_) {
    if (den < 0) {
        den = -den;
        num = -num;
    }
}

Frac() : Frac(0, 1) {}

Frac(T num_) : Frac(num_, 1) {}

explicit operator double() const {
    return 1. * num / den;
}

Frac &operator+=(const Frac &rhs) {
    num = num * rhs.den + rhs.num * den;
    den *= rhs.den;
    return *this;
}

Frac &operator-=(const Frac &rhs) {
    num = num * rhs.den - rhs.num * den;
    den *= rhs.den;
    return *this;
}

Frac &operator*=(const Frac &rhs) {
    num *= rhs.num;
    den *= rhs.den;
    return *this;
}

Frac &operator/=(const Frac &rhs) {
    num *= rhs.den;
    den *= rhs.num;
    if (den < 0) {
        num = -num;
        den = -den;
    }
    return *this;
}

friend Frac operator+(Frac lhs, const Frac &rhs) {
    return lhs += rhs;
}

friend Frac operator-(Frac lhs, const Frac &rhs) {
    return lhs -= rhs;
}

friend Frac operator*(Frac lhs, const Frac &rhs) {
    return lhs *= rhs;
}

friend Frac operator/(Frac lhs, const Frac &rhs) {
```

```
        return lhs /= rhs;
    }

    friend Frac operator-(const Frac &a) {
        return Frac(-a.num, a.den);
    }

    friend bool operator==(const Frac &lhs, const Frac &rhs) {
        return lhs.num * rhs.den == rhs.num * lhs.den;
    }

    friend bool operator!=(const Frac &lhs, const Frac &rhs) {
        return lhs.num * rhs.den != rhs.num * lhs.den;
    }

    friend bool operator<(const Frac &lhs, const Frac &rhs) {
        return lhs.num * rhs.den < rhs.num * lhs.den;
    }

    friend bool operator>(const Frac &lhs, const Frac &rhs) {
        return lhs.num * rhs.den > rhs.num * lhs.den;
    }

    friend bool operator<=(const Frac &lhs, const Frac &rhs) {
        return lhs.num * rhs.den <= rhs.num * lhs.den;
    }

    friend bool operator>=(const Frac &lhs, const Frac &rhs) {
        return lhs.num * rhs.den >= rhs.num * lhs.den;
    }

    friend std::ostream &operator<<(std::ostream &os, Frac x) {
        T g = std::gcd(x.num, x.den);
        if (x.den == g) {
            return os << x.num / g;
        } else {
            return os << x.num / g << "/" << x.den / g;
        }
    }

    std::string to_string() {
        T g = std::gcd(num, den);
        if (den == g) {
            return std::to_string(num) + "/" + std::to_string(g);
        } else {
            return std::to_string(num / g) + "/" + std::to_string(den / g);
        }
    }
};
```

### 1.11 基姆拉尔森公式

```
/**    基姆拉尔森公式（计算某年某月某日是星期几）
 *     2023-09-05: https://qoj.ac/submission/164735
**/
const int d[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

```
bool isLeap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}

int daysInMonth(int y, int m) {
    return d[m - 1] + (isLeap(y) && m == 2);
}

int getDay(int y, int m, int d) {
    int ans = 0;
    for (int i = 1970; i < y; i++) {
        ans += 365 + isLeap(i);
    }
    for (int i = 1; i < m; i++) {
        ans += daysInMonth(y, i);
    }
    ans += d;
    return (ans + 2) % 7 + 1;
}
```

### 1.12 德州扑克

```
namespace holdem {
    enum Suit {
        HighCard,           // 高牌
        Pair,               // 对子
        TwoPair,            // 两对
        ThreeOfAKind,       // 三条
        Straight,           // 顺子
        Flush,              // 同花
        FullHouse,          // 葫芦
        FourOfAKind,        // 铁支
        StraightFlush,      // 同花顺
        RoyalStraightFlush  // 同花大顺
    };
    enum CardType {
        Two = 0,
        Three = 1,
        Four = 2,
        Five = 3,
        Six = 4,
        Seven = 5,
        Eight = 6,
        Nine = 7,
        Ten = 8,
        Jack = 9,
        Queen = 10,
        King = 11,
        Ace = 12
    };
    enum Color {
        Heart = 0,    //
        Spade = 1,    //
        Club = 2,     //
        Diamond = 3   //
    };
    // 一张卡牌，用类型和颜色表示
```

```cpp
using Card = std::pair<CardType, Color>;
bool operator<(const Card &lhs, const Card &rhs) {
    return lhs.first < rhs.first;
}
// 手牌分数，用牌型和分数判断，分数只对相同牌型有效
using Score = std::pair<Suit, int>;
bool operator<(const Score &lhs, const Score &rhs) {
    return lhs.first < rhs.first or (lhs.first == rhs.first and lhs.second < rhs.second);
}
// 手牌
struct Hand {
    // 颜色，用位表示每张手牌是否存在
    std::array<short, 4> color{};
    // 数目，存放每种卡牌的数目
    std::array<short, 13> type{};
    void clear(){
        std::fill(color.begin(), color.end(), 0);
        std::fill(type.begin(), type.end(), 0);
    }
    void addCard(const Card& c){
        color[c.second] += (1 << c.first);
        type[c.first]++;
    }
    void removeCard(const Card& c){
        color[c.second] -= (1 << c.first);
        type[c.first]--;
    }
};
Score getType(const Hand &hand) {

    auto& colors = hand.color;
    auto& types = hand.type;
    // is RoyalStraightFlush
    for (short i = 0; i < 4; ++i) {
        if ((colors[i] & 0b111110000000000) == 0b111110000000000) {
            return {RoyalStraightFlush, 0};
        }
    }
    // is StraightFlush
    {
        int tv = 0b111110000000000;
        for (short i = Ace; i >= Five; --i) {
            for(int j = 0; j < 4; ++j){
                if((colors[j] & tv) == tv){
                    return {StraightFlush, i};
                }
            }
            tv >>= 1;
            if(__builtin_popcount(tv) < 5) tv |= (1 << 12);
        }
    }
    // is FourOfAKind
    {
        for (short i = Ace; i >= Two; --i) {
            if (types[i] >= 4) {
                for (short j = Ace; j >= Two; --j) {
```

```
                if (j == i) continue;
                if (types[j] > 0) {
                    return {FourOfAKind, i * 13 + j};
                }
            }
        }
    }
}
// is FullHouse
{
    for (short i = Ace; i >= Two; --i) {
        if (types[i] >= 3) {
            for (short j = Ace; j >= Two; --j) {
                if (j == i) continue;
                if (types[j] >= 2) {
                    return {FullHouse, i * 13 + j};
                }
            }
        }
    }
}
// is Flush
{
    int score = 0;
    for (short c = 0; c < 4; ++c) {
        if (__builtin_popcount(colors[c]) >= 5) {
            int tScore = 0;
            for (int j = Ace; j >= Two and __builtin_popcount(tScore) < 5; --j) {
                tScore |= (colors[c] & (1 << j));
            }
            score = std::max(score, tScore);
        }
    }
    if (score) return {Flush, score};
}
// is Straight
{
    for (int i = Ace; i >= Five; --i) {
        bool ok = true;
        for (int j = 0; j < 5 and ok; ++j) {
            int p = (i + 13 - j) % 13;
            if (types[p] == 0) {
                ok = false;
            }
        }
        if (ok) {
            return {Straight, i};
        }
    }
}
// is ThreeOfAKind
{
    for (int i = Ace; i >= Two; --i) {
        if (types[i] >= 3) {
            std::vector<int> others;
            for (int j = Ace; j >= Two and others.size() < 2; --j) {
```

```
                    if (types[j] > 0) {
                        others.push_back(j);
                    }
                }
                return {ThreeOfAKind, i * 169 + others[0] * 13 + others[1]};
            }
        }
    }
    // is TwoPair
    {
        std::vector<int> pairs;
        for (int i = Ace; i >= Two and pairs.size() < 2; --i) {
            if (types[i] >= 2) {
                pairs.push_back(i);
            }
        }
        if (pairs.size() >= 2) {
            for (int i = Ace; i >= Two; --i) {
                if (types[i] > 0) {
                    return {TwoPair, pairs[0] * 169 + pairs[1] * 13 + i};
                }
            }
        }
    }
    // is Pair
    {
        for (int i = Ace; i >= Two; --i) {
            if (types[i] >= 2) {
                int score = i;
                for (int j = Ace, cnt = 0; j >= Two and cnt < 3; --j) {
                    if (j == i) continue;
                    if (types[j] > 0) {
                        ++cnt;
                        score = score * 13 + j;
                    }
                }
                return {Pair, score};
            }
        }
    }
    // HighCard
    int highScore = 0;
    for (int i = Ace, cnt = 0; i >= Two and cnt < 5; --i) {
        if (types[i]) {
            highScore = highScore * 13 + i;
            ++cnt;
        }
    }
    return {HighCard, highScore};
}
char cardType2char(CardType ct) {
    return "23456789TJQKA"[ct];
}
CardType char2cardType(char c) {
    switch (c) {
        case '2':
```

```
                return Two;
            case '3':
                return Three;
            case '4':
                return Four;
            case '5':
                return Five;
            case '6':
                return Six;
            case '7':
                return Seven;
            case '8':
                return Eight;
            case '9':
                return Nine;
            case 'T':
                return Ten;
            case 'J':
                return Jack;
            case 'Q':
                return Queen;
            case 'K':
                return King;
            default:
                return Ace;
        }
    }
}
```